ATIVIDADE PRÁTICA

Sinais,DadoseTransmissãoDigital e Analógica com ESP32

Grupos de 3 alunos · Materiais:2×ESP32,potenciômetro10k Ω,LED+resistor 220 Ω, jumpers, protoboard

Aluno 1	Aluno 2	Aluno 3
Loren Pires de Oliveira Ferreira	Entoni Nascimento Carvalho	Thiago Paes de Barros Costa

Turma	Data	Nota
Engenharia da Computação	27/04/2026	

Objetivo

Observar e comparar na prática a transmissão digital (UART entre dois ESP32) e a transmissão analógica (PWM a partir de ADC), associando os resultados à teoria de sinais e dados vista em aula.

PARTE 1 — Fundamentos Teóricos (20 min)

Responda individualmente com o material de aula — sem internet.

1.1 Sinais e Dados

Classifique cada afirmação como V (verdadeira) ou F (falsa) e justifique em 1–2 linhas:

Afirmação	V / F	Justificativa
Um sinal analógico pode assumir infinitos valores dentro de uma faixa.	V	Sinais analógicos são contínuos no tempo e na amplitude, podendo variar infinitamente entre um valor mínimo e máximo.
Dados digitais só podem representar os valores 0 e 1.	F	Embora usem lógica binária na base, dados digitais podem representar qualquer valor discreto (números, letras) através de codificação.
A codificação NRZ-L usa variação de tensão para indicar transições de bit.	F	No NRZ-L o nível de tensão é determinado pelo valor do bit (ex: alto para 0, baixo para 1), e não por transições.
Em Manchester, cada bit tem sempre uma transição no meio do intervalo.	V	A transição central é a característica principal do Manchester, servindo tanto para o sincronismo do clock quanto para definir o valor do bit.
Um sinal de áudio capturado por microfone é um exemplo de sinal digital.	F	O som capturado pelo microfone é uma onda de pressão convertida em sinal elétrico analógico; ele só se torna digital após passar por um conversor (ADC).

1.2 Transmissão Digital vs. Analógica

Complete a tabela com suas próprias palavras:

Característica	Transmissão Digital	Transmissão Analógica
Forma do sinal	Composta por pulsos discretos (descontínuos), geralmente representados por níveis de tensão que definem estados binários.	Composta por ondas eletromagnéticas contínuas que variam suavemente em amplitude e frequência ao longo do tempo.
Sensibilidade a ruído	Baixa. O receptor só precisa identificar se o sinal está acima ou abaixo de um limiar para entender o dado, sendo mais imune a interferências.	Alta. Como qualquer pequena variação na onda transporta informação, o ruído é amplificado junto com o sinal, causando distorção.
Exemplo no cotidiano	Streaming de vídeo (Netflix/YouTube), conexões Wi-Fi ou transferência de arquivos via USB.	Rádio FM/AM tradicional, chamadas de voz em telefonia fixa antiga ou discos de vinil.
Facilidade de regeneração	Alta. O sinal pode ser totalmente reconstruído (repetidores limpam o ruído e retransmitem o pulso original) sem perda de qualidade.	Baixa. Amplificadores aumentam o sinal, mas também aumentam o ruído acumulado no percurso, degradando a qualidade progressivamente.

PARTE 2 — Experimento A: Transmissão Digital UART (30 min)

2.1 Hardware

Ligue os dois ESP32 conforme a tabela. Conecte o GND primeiro.

ESP32 Transmissor (A)	Direção	ESP32 Receptor (B)
GPIO 17 (TX2)	→	GPIO 16 (RX2)
GPIO 16 (RX2)	←	GPIO 17 (TX2)
GND	—	GND

> ⚠ **Conecte GND–GND antes de qualquer outro fio. Pinos de dados sem GND em comum podem danificar as placas.**

2.2 Código — Transmissor

```
// transmissor.ino
#define BAUD_RATE 9600
#define  PINO_TX  17
#define  PINO_RX  16
void setup() {

  Serial.begin(115200);
  Serial2.begin(BAUD_RATE, SERIAL_8N1, PINO_RX, PINO_TX);
  Serial.println("Transmissor pronto.");
}

void loop() {
  String mensagem = "DADO:" + String(millis());  // Ex: DADO:3042
 Serial2.println(mensagem);
  Serial.println("Enviado: " + mensagem);
  delay(1000);
}
```

2.3 Código — Receptor

```
// receptor.ino
#define BAUD_RATE 9600
#define  PINO_TX  17
#define  PINO_RX  16
#define  LED_PIN   2
void setup() {

  Serial.begin(115200);
  Serial2.begin(BAUD_RATE, SERIAL_8N1, PINO_RX, PINO_TX);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("Receptor pronto.");
}

void loop() {
  if (Serial2.available()) {
    String recebido = Serial2.readStringUntil('\n');
    Serial.println("Recebido: " + recebido);
    digitalWrite(LED_PIN, HIGH); delay(100); digitalWrite(LED_PIN, LOW);
  }
}
```

2.4 Questões de observação

a) Qual o BAUD_RATE em uso? O que acontece se o transmissor usar 9600 e o receptor 115200? Explique o motivo.

baud_rate 9600. Se usar diferente os dois vão perder sincronismo e mensagem vai corromper, pois emissor estará enviando mais lento e o receptor estará lendo mais rapido.

b) Copie 3 mensagens recebidas exatamente como aparecem no Monitor Serial do receptor:

Recebido:□□□

c) SERIAL_8N1: 8 bits de dados, sem paridade, 1 bit de parada. Para que serve o bit de parada em transmissão assíncrona?

O bit de parada (stop bit) serve para indicar o fim do frame e permitir a re-sincronização do receptor.

d) Desconecte o fio TX → RX enquanto o sistema roda. Descreva o que aconteceu no receptor e explique por quê.

Com a comunicação interrompida entre TX e RX, o receptor para de receber sinal e então para de exibir a mensagem de recebimento de dados.

PARTE 3 — Experimento B: Sinal Analógico com Potenciômetro e PWM (30 min)

3.1 Hardware

Terminal do potenciômetro	Conectar em
Terminal esquerdo (extremidade)	3V3
Terminal central (cursor)	GPIO 34 (ADC)
Terminal direito (extremidade)	GND
Anodo do LED (via resistor 150 Ω)	GPIO 25 (PWM)
Catodo do LED	GND

3.2 Código — ADC e PWM

```
// analogico_pwm.ino
#define   PINO_POT  34
#define   PINO_LED  25
#define  CANAL_PWM   0
#define   FREQ_PWM 1000   // Hz
#define RESOLUCAO     8   // bits → 0 a 255

void setup() {
  Serial.begin(115200);
  ledcSetup(CANAL_PWM, FREQ_PWM, RESOLUCAO);
  ledcAttachPin(PINO_LED, CANAL_PWM);
}

void loop() {
  int leitura = analogRead(PINO_POT);          // 0 a 4095 (12 bits)
  int pwm     = map(leitura, 0, 4095, 0, 255); // remapeia para 8 bits
  ledcWrite(CANAL_PWM, pwm);
  float tensao = leitura * (3.3 / 4095.0);
  Serial.print("ADC: ");    Serial.print(leitura);
  Serial.print(" | V: ");   Serial.print(tensao, 2);
  Serial.print(" | PWM: "); Serial.println(pwm);
  delay(200);
}
```

3.3 Coleta de dados

Gire o potenciômetro a 5 posições distintas e preencha:

Posição	ADC (0–4095)	Tensão (V)	PWM (0–255)	Duty Cycle (%)
1	4095	3.30	255	100%
2	2801	2.26	174	68,2%
3	2013	1.62	125	49%
4	1424	1.15	88	34,5%
5	118	0.10	7	2,7%

3.4 Análise

a) O sinal no GPIO 34 é analógico ou digital? E o sinal no GPIO 25? Justifique a diferença.

O sinal no GPIO 34 (ADC) é analógico, pois recebe tensão que varia continuamente entre 0 e 3,3V conforme o potenciômetro é girado. Já o sinal GPIO 25 (PWM) é uma onda quadrada que alterna entre 0V e 3,3V em alta frequência. A diferença é a natureza do sinal, um é contínuo (analógico) e o outro discreto (digital)

b) O ADC tem resolução de 12 bits. Quantos níveis distintos ele reconhece? Qual a menor variação de tensão detectável (0–3,3 V)?

Com uma resolução de 12 bits, o ADC reconhece 2^{12} = 4096 níveis diferentes (de 0 a 4095). Como a faixa vai de 0 a 3,3V, a menor variação de tensão detectável é 3,3 / 4095 que é aproximadamente 0,806 mV. Abaixo disso, o ADC não vai conseguir distinguir dois valores diferentes

c) O PWM não é um sinal analógico, mas imita um. Explique como o duty cycle simula diferentes níveis de tensão e por que isso controla o brilho do LED.

O PWM funciona ligando e desligando o sinal digital rapidamente (1000 vezes por segundo na nossa atividade). O duty cycle representa a porcentagem de tempo que o sinal fica em 3,3V. Se o duty cycle é 50%, o led fica ligado na metade do tempo e desligado na outra metade. Quanto maior o duty cycle, mais tempo o led fica ligado e mais brilhante parece. Isso simula diferentes níveis de tensão média sem que o sinal seja analógico

PARTE 4 — Integração e Desafio (10 min)

Desafio

Combine os dois experimentos. Leia o potenciômetro no Transmissor, envie o valor ADC via UART ao Receptor e controle o brilho do LED no Receptor proporcionalmente ao valor recebido.

4.1 Planejamento — esboce o código antes de implementar

Transmissor — o que precisa mudar no loop() da Parte 2 para enviar o valor ADC em vez de millis()?

Foi necessário substituir a geração da mensagem baseada em millis() pela leitura do ADC (analogRead) e enviar esse valor.

Receptor — como extrair o inteiro da string recebida e usá-lo para controlar o PWM?

O valor recebido como string é convertido para inteiro com toInt(), depois ajustado para a faixa de PWM usando map() e aplicado ao LED com ledcWrite().

4.2 Resultado

Funcionou? Descreva o comportamento. Se não, descreva o sintoma e sua hipótese sobre a causa.

Sim. O transmissor envia para o receptor o valor obtido em analogRead que vem do potenciometro. O receptor recebe a string e converte para Int o valor ADC, e envia PWM para controlar a potencia do led.

PARTE 5 — Reflexão Final (individual, sem consulta)

1. Qual é a diferença fundamental entre sinal e dado? Dê um exemplo de cada a partir desta aula.

O dado é a informação, já o sinal é o que vai transmitir essa informação.
Exemplo: Dado: o número binário 1011
Sinal: uma sequência de pulsos elétricos (ou ondas) que representam esse 1011 passando por um cabo ou pelo ar.

2. Em qual dos dois experimentos a informação é mais suscetível a ruído elétrico? Por quê?

A informação é mais suscetível a ruído elétrico no Experimento 2. Pois, no segundo experimento, o sinal do potenciômetro é analógico, ou seja, ele pode assumir infinitos valores dentro de uma faixa contínua de tensão, o que gera interferências e acumulos.

3. Para transmitir a leitura do potenciômetro por 100 metros, o que seria mais confiável: enviar o sinal analógico diretamente ou digitalizar e enviar via UART? Justifique.

O mais confiável é digitalizar o sinal e enviar via UART, e não transmitir o sinal analógico diretamente. Pois, o sinal analógico é mais sensível a ruidos, quedas de tensão e distorções acumuladas

⚠ **Respostas copiadas de IA ou de colegas resultam em nota zero na questão. As questões de análise exigem dados coletados na bancada — valores que diferem do experimento real são facilmente identificáveis.**